# Graph Theory Coding Task (Week 6–8)

Algorithm Report: TSP, Chinese Postman Problem, and Knight’s Tour

## Team Identity

| No | Name | Student ID | Role |
|---|---|---|---|
| 1 | Padhang Abiyu Fikri | 5025251014 | Task 1 |
| 2 | Dzulfiqar Rafi'ussunnah | 5025251011 | Task 2 |
| 3 | Aditya Lingga Mardika | 5025251158 | Task 3 |
| 4 | Muhammad Faris Alfarrel | 5025251002 | Report / Readme |

**Repository:** https://github.com/Gaerowes/Graph-Teory-Assignment3-Group-3

# 1. Overview

This report explains the algorithm idea, implementation, and computational complexity for three simulators: the SPPG delivery route (Travelling Salesman Problem), the DLH garbage-collection route (Chinese Postman Problem), and the Knight’s Tour. Tasks 1 and 2 run on the real Surabaya road network; Task 3 runs on a configurable chessboard.

## 1.1 Shared design

- **Language and libraries:** Python 3; OSMnx and NetworkX for the road graph; Streamlit with Folium for the interactive map.
- **Road network:** downloaded once from OpenStreetMap (drive network of Surabaya) and cached as GraphML. It is a directed multigraph (MultiDiGraph), so one-way streets are respected. Edge length (metres) is the cost; travel time uses the OSM speed attribute or a fixed average speed.
- **Notation:** V and E are the number of nodes and edges of the (sub)graph, k is the number of schools inside the radius, N is the number of chessboard squares.
- **Radius filter:** For Task 2, x km is measured as a straight-line radius from the clicked base location. Major road segments that intersect the circle are selected in full, and the base is snapped to the nearest drivable road node

# 2. Task 1: Travelling Salesman Problem (SPPG Simulator)

## 2.1 Problem

Starting at the SPPG, visit every school within x km exactly once and return to the SPPG, following the actual one-way road network. Road segments may be reused when travelling between schools.

## 2.2 Algorithm idea

The road network is not a complete graph, so the problem is reduced in two stages (metric closure):

1. **Build the distance matrix.** Run Dijkstra from the SPPG and from each school node. This yields the shortest road distance d(i, j) between every pair of the k+1 points. Because roads are directed, d(i, j) ≠ d(j, i) in general (asymmetric TSP).
2. **Solve TSP on the matrix.** Find the minimum-cost Hamiltonian cycle over the k+1 points using the matrix costs.
3. **Expand the route.** Replace each matrix leg by its stored shortest road path to obtain the polyline drawn on the map.

**Solver choice.** For small k (about 15–20 or fewer) the exact Held-Karp dynamic programme is used. For larger k the program falls back to a heuristic: nearest-neighbour construction followed by Or-opt local search. Or-opt (moving a segment without reversing it) is used instead of 2-opt because reversing a segment changes the cost on an asymmetric matrix.

## 2.3 Held-Karp dynamic programme

Let dp[S][j] be the minimum cost of a path that starts at the SPPG, visits exactly the set S of schools, and ends at school $j \in S$.

```
dp[{j}][j]  = d(0, j)
dp[S][j]   = min over i in S\{j} of  dp[S\{j}][i] + d(i, j)
answer     = min over j of  dp[all][j] + d(j, 0)
```

Subsets are encoded as bitmasks; parent pointers are stored to reconstruct the visiting order.

## 2.4 Implementation outline

- load_graph(): load or download and cache the Surabaya MultiDiGraph.
- load_schools(): query OSM for amenity=school, snap each to its nearest node.
- select_schools(base, x): keep schools inside the radius.
- distance_matrix(): Dijkstra from each point, keeping predecessor paths.
- held_karp() / or_opt(): the two solvers.
- Streamlit UI: click to set the SPPG, slider for x, then show the route overlay, visiting order, total distance, estimated time, and solver execution time (measured with a high-resolution timer around the solver only).

## 2.5 Complexity

| Step | Time | Space |
|---|---|---|
| Dijkstra from k+1 sources | $O((k+1)(E + V \log V))$ | $O(V + E)$ per run; $O(k^2)$ matrix |
| Held-Karp (exact) | $O(k^2 \cdot 2^k)$ | $O(k \cdot 2^k)$ |
| Nearest neighbour | $O(k^2)$ | $O(k)$ |
| Or-opt, one pass | $O(k^2)$ moves, $O(1)$ delta each | $O(k)$ |
| Route expansion | O(total path length) | O(total path length) |

For realistic k, the dominant cost is either the Dijkstra phase (large k) or Held-Karp (k up to about 20). TSP is NP-hard, so no polynomial exact algorithm is known; the heuristic gives no optimality guarantee on an asymmetric instance.

# 3. Task 2: Chinese Postman Problem (DLH Simulator)

## 3.1 Problem

Find a closed route that starts and ends at the dump-truck base and traverses the trunk, primary, secondary, and tertiary road segments within x km. The road network is directed, so one-way streets are followed in the correct direction. Road segments without a directed route to and from the base may be omitted.

## 3.2 Algorithm idea

1. Download Surabaya's drivable road network from OpenStreetMap using OSMnx and project it to a coordinate system measured in metres.
2. Select major road segments that intersect a circle of radius x km around the chosen base. A segment crossing the circle's boundary is included in full.
3. Snap the selected base location to the nearest road node. Keep roads in the same strongly connected component as that node, so the route can reach them and return.
4. If the selected roads are disconnected from each other, connect each group to the base with shortest paths through the drivable road network. These connecting paths may leave the radius.
5. Balance the directed graph using minimum-cost flow. The flow finds the extra road traversals needed to make every node's in-degree equal its out-degree. Add those traversals to the graph.
6. Find an Eulerian circuit starting at the base. This gives a closed route that covers every selected road segment.

The balancing step finds the minimum additional distance for the connected graph. The connecting paths are selected with a heuristic, so the entire route is not guaranteed to be globally shortest.

## 3.3 Implementation outline

- load_city(): download and cache Surabaya's drivable road network.
- get_roads(): select major roads within the radius, snap the base, and connect the selected roads to the base.
- solve_cpp(): balance the graph with minimum-cost flow and find an Eulerian circuit.
- prepare_output(): create the route coordinates and street traversal list.
- Streamlit interface: select the base on the map, set the radius and average speed, and view the route, street order, distance, estimated travel time, and solver time.

## 3.4 Complexity

| Step | Time | Space |
|---|---|---|
| Load and filter road network | O(V + E) | O(V + E) |
| Find connected components | O(V + E) | O(V + E) |
| Connect selected road groups | O(C (E + V log V)) | O(V + E) |
| Minimum-cost flow | Depends on the network and solver | O(V + E) |
| Eulerian circuit | O(E′) | O(E′) |

Here, V and E are the number of nodes and directed road segments in the downloaded network, C is the number of selected road groups, and E′ is the number of segments after repeated traversals are added. NetworkX's minimum-cost flow runtime depends on the graph, so a single simple bound is not given.

## 3.5 Assumptions and edge cases

- The radius is measured in a straight line from the clicked base location.
- Road segments that cross the radius boundary are included in full.
- The clicked location is snapped to the nearest drivable road node.
- Connecting paths may leave the radius.
- Travel time is estimated using the average speed selected in the interface. It does not account for traffic, stops, or loading.
- The road network follows general driving directions in OpenStreetMap and may not include every truck-specific restriction.

# 4. Task 3: Knight's Tour

## 4.1 Problem

Given a board of C columns and R rows and a starting square, output a sequence of knight moves that visits every square exactly once. If no such path exists, print "path not found". Coordinates are (column, row), 0-indexed, as in the assignment grid.

## 4.2 Algorithm idea

Depth-first backtracking over the knight-move graph, guided by Warnsdorff's rule: from the current square, try the unvisited neighbouring squares in ascending order of their own number of onward unvisited moves. This visits hard-to-reach squares (corners and edges) first and usually finds a tour with little or no backtracking.

- **Tie-breaking:** ties are broken deterministically (for example by distance to the board centre or by move order) so results are reproducible.
- **Early rejection:** if C × R is odd and (column + row) of the start is odd, no tour exists (colour-class counting argument), so the program prints "path not found" immediately.
- **Termination:** backtracking is exhaustive, so "path not found" is exact; a node-count limit can be set for very large boards and, if reached, is reported as such.

## 4.3 Implementation outline

```
solve(C, R, sx, sy):
  if C*R odd and (sx+sy) odd: return NOT_FOUND
  visited[sx][sy] = true; path = [(sx, sy)]
  return dfs(sx, sy, 1)
dfs(x, y, count):
  if count == C*R: return true
  for (nx, ny) in unvisited_moves(x, y) sorted by onward_degree:
        mark, push; if dfs(nx, ny, count+1): return true; unmark, pop
  return false
```

Input is two lines: "C R" and "sx sy". The output is the ordered list of coordinates, or "path not found". An iterative stack or a raised recursion limit is needed for large boards.

### 4.4 Complexity

| Case | Time | Space |
|---|---|---|
| Warnsdorff, no backtracking | O(N) (each step inspects at most 8 moves with at most 8 degree checks) | O(N) |
| Worst case with backtracking | O(8^N) (exponential) | O(N) recursion depth and visited array |

Here $N = C \times R$. In practice Warnsdorff ordering makes the common cases near-linear; the exponential bound only applies to boards with no tour or with an unlucky start, where the search must be exhausted or cut off.

## 5. Complexity Summary

| Task | Algorithm | Dominant complexity |
|---|---|---|
| TSP (small k) | Dijkstra + Held-Karp | $O(k(E + V \log V) + k^2 2^k)$ |
| TSP (large k) | Dijkstra + nearest neighbour + Or-opt | $O(k(E + V \log V) + \text{passes} \cdot k^2)$ |
| CPP (directed) | Strongly connected components + minimum-cost flow + Eulerian circuit | $O(C(E + V \log V) + \text{flow} + E')$ |
| Knight's Tour | Warnsdorff + backtracking | O(N) typical, O(8^N) worst |

## 6. AI Usage Disclosure

AI assistance (Claude) was used for planning the approach and drafting this report.